

UNIVERSIDAD MAYOR REAL Y PONTIFICIA
SAN FRANCISCO XAVIER DE CHUQUISACA
FACULTAD DE TECNOLOGÍA



TEMA: GRANJA AVÍCOLA

Nombres: Quispe Sucullani Jose David

Gutierrez Murillo Gerzon

Carrera: Ingeniería de Ciencias de la computación

Materia: Desarrollo de aplicaciones móviles

Docente: Velasquez Aroni Oswaldo Gerardo

Sucre-Bolivia

ÍNDICE

1. Introducción	4
El proyecto Gallinero es un sistema de monitoreo inteligente diseñado para la gestión y el control automatizado de un gallinero. Combina hardware, un script de conexión y una aplicación móvil para ofrecer una solución completa.	4
Los componentes principales del sistema son:	4
Aplicación Móvil (dispenser_app): Una interfaz de usuario desarrollada en Flutter con un tema oscuro que actúa como el panel de control principal. Permite a los usuarios monitorear y gestionar los aspectos clave del gallinero, incluyendo:	4
○ Funcionalidades de Monitoreo y Control: Acceso directo a la gestión de Agua, Comida, Cámara y Luces.	4
○ Monitoreo Ambiental: Control y visualización de la Temperatura.	4
○ Navegación Adicional: Un menú lateral brinda acceso a pantallas de Estadísticas y Detecciones.	4
Sistema de Comunicación y Control (Python/Arduino): Un script en Python (arduino api.py) gestiona la conexión bidireccional entre el sistema de control físico (probablemente una placa Arduino, a través del puerto serial COM4) y una API de servidor.	4
Flujo de Datos: El script lee datos de sensores (en formato JSON) provenientes del hardware y los envía a un servidor (mediante una API PHP).	4
Control Remoto: Utiliza long polling para verificar constantemente si hay cambios en el estado de los actuadores (como las Luces) en el servidor y, de ser necesario, envía comandos de control (LUCES=1\n o LUCES=0\n) al hardware para ejecutar la acción.	4
Sistema de Detección/Cámaras: El sistema incluye una función de Detecciones que se integra con un servidor. Una pantalla dedicada en la aplicación móvil se encarga de conectar con el backend para listar y mostrar las imágenes capturadas y guardadas por el sistema de detección (presumiblemente anomalías o eventos importantes).	4
2. objetivo	5
3. Materiales Usados	6
4. Software Usado	6
4.1. Software para la Aplicación Móvil (Frontend)	6
4.2. Software para el Servidor Central (Backend y Base de Datos)	7
4.3. Software para la Conexión y Control de Hardware	7
5. Github	8
6. Comandos utilizados	8
6.1. Flutter (Aplicación Móvil dispenser_app)	8
6.2. Python (Script de Conexión arduinoapi.py)	8
6.3. Arduino (Hardware y Microcontrolador)	9
6.4. PHP (API del Servidor - Backend)	9
7. GRÁFICA DE LAS CONEXIONES	10
8. Gráficas y resultados del modelo	11
8.1 Curva de aprendizaje	11
8.2 Matriz de confusión	11
8.3 Labels	12
8.4. Resultados de train test y valid	12

8.5 Prueba de imagen	13
9. Fotografías de la maqueta	14
9.1 Prueba de foco	14
9.2. Prueba de comida	15
9.3. Bomba de agua	16
9.4 Ventilador	16
9.5 Temperatura	17
10. Capturas de la web	17
10.1. Home	17
10.2. Comedero	18
10.3. Bebedero	18
10.4. Luz	19
10.5. Temperatura	19
10.6. Notificaciones	20
10.7. Cámara	21
10.8. Detecciones	21
11. Conexiones	22
11.1. Conexión del sensor DHT11 (Temperatura / Humedad)	22
11.2. Conexión del sensor ultrasónico HC-SR04	22
11.3. Control de la mini bomba de agua (5V)	22
11.4. Control del ventilador de 5V	22
11.5. LED indicador (opcional)	23
11.6. Potenciómetro o sensor lineal (se ve en tu imagen superior)	23
11.7. Alimentación y GND	23
11.8. Funcionamiento General	23
11. Conclusión	24
12. Bibliografía	24

1. Introducción

El proyecto **Gallinero** es un sistema de monitoreo inteligente diseñado para la gestión y el control automatizado de un gallinero. Combina hardware, un *script* de conexión y una aplicación móvil para ofrecer una solución completa.

Los componentes principales del sistema son:

Aplicación Móvil (dispenser_app): Una interfaz de usuario desarrollada en Flutter con un tema oscuro que actúa como el panel de control principal. Permite a los usuarios monitorear y gestionar los aspectos clave del gallinero, incluyendo:

- **Funcionalidades de Monitoreo y Control:** Acceso directo a la gestión de **Agua, Comida, Cámara y Luces**.
- **Monitoreo Ambiental:** Control y visualización de la **Temperatura**.
- **Navegación Adicional:** Un menú lateral brinda acceso a pantallas de **Estadísticas y Detecciones**.

Sistema de Comunicación y Control (Python/Arduino): Un *script* en Python (`arduino_api.py`) gestiona la conexión bidireccional entre el sistema de control físico (probablemente una placa Arduino, a través del puerto serial COM4) y una API de servidor.

Flujo de Datos: El *script* lee datos de sensores (en formato JSON) provenientes del hardware y los envía a un servidor (mediante una API PHP).

Control Remoto: Utiliza *long polling* para verificar constantemente si hay cambios en el estado de los actuadores (como las **Luces**) en el servidor y, de ser necesario, envía comandos de control (`LUCES=1\n` o `LUCES=0\n`) al hardware para ejecutar la acción.

Sistema de Detección/Cámaras: El sistema incluye una función de **Detecciones** que se integra con un servidor. Una pantalla dedicada en la aplicación móvil se encarga de conectar con el *backend* para listar y mostrar las imágenes capturadas y guardadas por el sistema de detección (presumiblemente anomalías o eventos importantes).

2. objetivo

El objetivo principal del proyecto **Gallinero** es establecer un **Sistema de Monitoreo y Control Inteligente** para gestionar de manera remota y automatizada las condiciones esenciales dentro de un gallinero, asegurando así el bienestar de las aves y optimizando su manejo.

Este objetivo se desglosa en metas específicas, según los componentes del sistema:

1. **Monitoreo y Reporte de Datos Ambientales y de Recursos:** Recolectar continuamente datos críticos del ambiente (como la **Temperatura**) y de los recursos (niveles de **Agua** y **Comida**), enviándolos desde el *hardware* (Arduino/puerto serial) a una API central de servidor.
2. **Control Remoto de Actuadores:** Permitir que el usuario controle funciones clave del gallinero, como encender o apagar las **Luces**, mediante la aplicación móvil. El sistema está diseñado para sincronizar este estado de control a través de una API en el servidor, enviando los comandos correspondientes al *hardware*.
3. **Vigilancia Visual y Detección de Eventos:** Mantener vigilancia del entorno a través de la **Cámara** y almacenar registros de **Detecciones** (probablemente eventos o anomalías capturadas) en el servidor. La aplicación móvil permite a los usuarios acceder y visualizar esta lista de imágenes guardadas.
4. **Visualización de la Información Operacional:** Proporcionar un **Panel principal** accesible desde la aplicación móvil que muestre de forma clara el estado del sistema y permita a los usuarios navegar a pantallas de **Estadísticas** y **Detecciones** para un análisis más profundo.

3. Materiales Usados



4. Software Usado

El proyecto **Gallinero** requiere varios componentes de software para funcionar, divididos en tres áreas principales: la aplicación móvil (frontend), el servidor web (backend/API) y la capa de comunicación con el hardware.

El software necesario es:

4.1. Software para la Aplicación Móvil (Frontend)

La aplicación de monitoreo (dispenser.app) fue desarrollada utilizando el siguiente *framework* y herramientas:

- **Flutter SDK**: El *framework* de desarrollo necesario para compilar, ejecutar y mantener la aplicación móvil.

- **Dart:** El lenguaje de programación utilizado por Flutter.
- **Gestor de Dependencias de Flutter:** Requiere la instalación de paquetes como `http` para realizar las peticiones a la API del servidor.
- **IDE/Editor de Código:** Un entorno como Visual Studio Code o Android Studio con soporte para Flutter.

4.2. Software para el Servidor Central (Backend y Base de Datos)

El servidor es fundamental para almacenar los datos de los sensores y el estado de los actuadores (como las luces), actuando como intermediario entre la aplicación móvil y el hardware.

- **Servidor Web:** Un servidor HTTP (como Apache o Nginx), necesario para alojar y ejecutar los archivos de la API.
- **PHP:** El lenguaje de programación en el que están escritos los scripts de la API (`nuevo_api.php`, `listar_imagenes.php`, etc.), que gestionan las peticiones de la aplicación y la comunicación con la base de datos.
- **Sistema de Gestión de Bases de Datos (SGBD):** Probablemente **MySQL** o similar, necesario para almacenar los datos de los sensores, ya que existe un archivo de script de base de datos (`sensoresdb.sql`).

4.3. Software para la Conexión y Control de Hardware

Un *script* de Python actúa como un puente entre la API del servidor y la placa de control física.

- **Intérprete de Python:** Para ejecutar el script de comunicación (`arduinoapi.py`).
- **Librerías de Python:**
 - `pyserial` (o `serial`): Librería crucial para la comunicación con el hardware (Arduino) a través del puerto serial (ej. COM4).
 - `requests`: Librería para realizar peticiones HTTP y comunicarse con la API del servidor.
- **Controladores del Puerto Serial:** Drivers instalados en el sistema operativo para permitir que la computadora reconozca y se comunique con el puerto (USB) del hardware de control.
- **IDE de Arduino:** Entorno necesario si se requiere modificar o cargar el código (`sketch_nov30.ino`) con el nombre la cual se encuentra en el microcontrolador.

5. Github

<https://github.com/GersonGuMu/Gallinero.git>

6. Comandos utilizados

Detalle de la configuración y los comandos esenciales para cada componente del *software* del proyecto **Gallinero**.

6.1. Flutter (Aplicación Móvil `dispenser_app`)

El objetivo es preparar y lanzar la interfaz de usuario de monitoreo. Para lograrlo, los comandos de Flutter se enfocan en la gestión de las dependencias de la aplicación y su ejecución:

- **Instalación de Dependencias:** El comando principal es **flutter pub get**, el cual debe ser ejecutado dentro del directorio `dispenser_app`. Este paso descarga e integra todas las bibliotecas de Dart necesarias, como el paquete `http`, esencial para que la aplicación realice peticiones a la API del servidor.
- **Ejecución de la Aplicación:** Una vez que las dependencias estén listas, el comando **flutter run** inicia la aplicación en el dispositivo o emulador configurado. Si el objetivo es un navegador web, se utiliza el comando **flutter run -d chrome** para lanzar la aplicación directamente en Chrome.

6.2. Python (Script de Conexión `arduinoapi.py`)

El *script* de Python es el intermediario crucial entre la electrónica física (Arduino) y el servidor web.

- **Instalación de Librerías:** Se requiere instalar dos librerías principales utilizando el gestor de paquetes pip:
 - **pip install pyserial** (o `pip install serial`): Permite la comunicación directa a través del puerto serial (COM4) para enviar y recibir datos del Arduino.
 - **pip install requests:** Facilita la comunicación HTTP, lo que permite al *script* enviar las lecturas de los sensores al servidor (POST) y consultar el estado deseado de los actuadores (GET, para el control de luces).
- **Ejecución del Script:** El comando **python arduinoapi.py** pone en marcha el sistema de comunicación. El *script* entra en un bucle continuo (`while True`) para leer la data JSON del Arduino y subirla a la API del servidor, al mismo tiempo que verifica si ha habido un cambio en el estado de las luces en el servidor para enviar el comando de control correspondiente (`LUCES=1\n` o `LUCES=0\n`) al Arduino.

6.3. Arduino (Hardware y Microcontrolador)

En lugar de comandos de consola, se trata de acciones a través del Entorno de Desarrollo Integrado (IDE) de Arduino:

- **Acción de "Subir" (*Upload*):** Se utiliza el **IDE de Arduino** para compilar el código fuente (el *sketch*, como `sketch_nov30.ino`) y cargarlo en la placa de microcontrolador. Este código contiene la lógica para la lectura de sensores y el manejo de los actuadores.
- **Configuración del Puerto Serial:** Se debe verificar que el código en el Arduino y el script Python usen la misma tasa de baudios (**9600**) para establecer una comunicación serial estable.

6.4. PHP (API del Servidor - Backend)

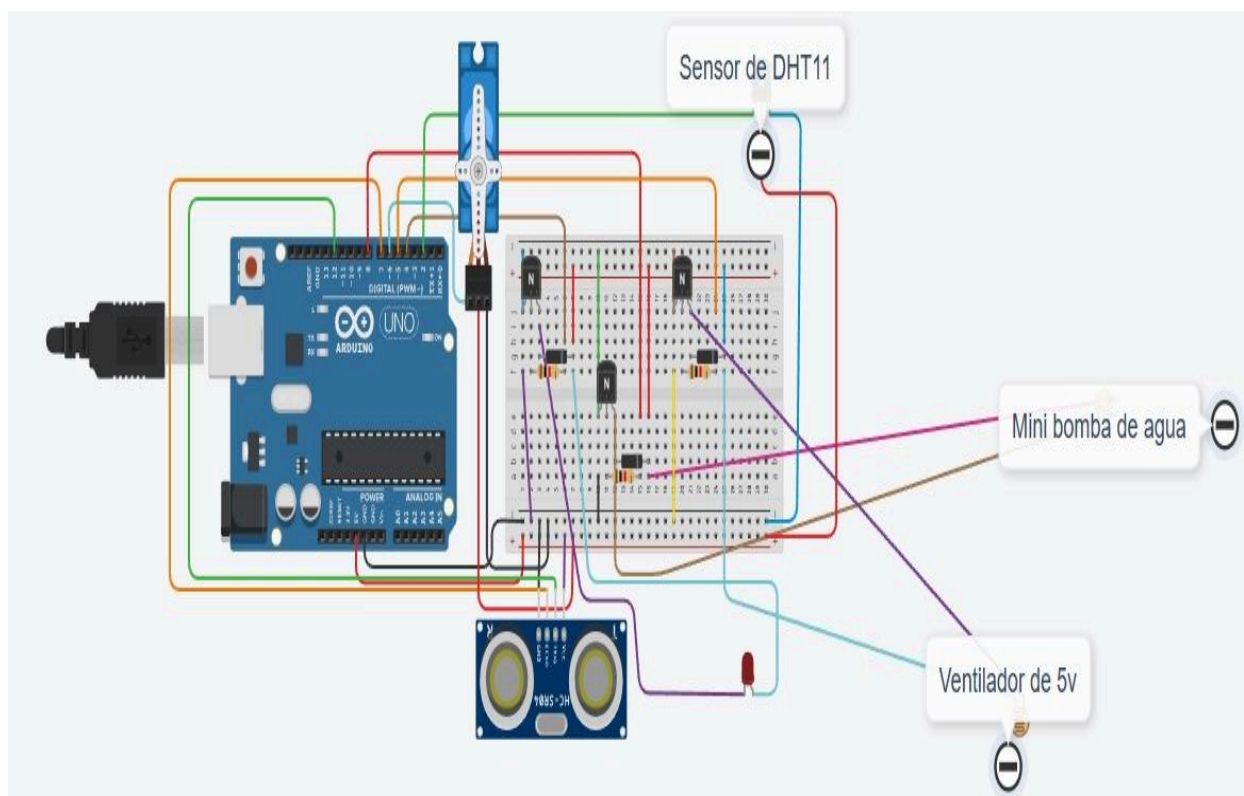
El código PHP es ejecutado por un servidor web, por lo que el enfoque está en la configuración del entorno para que las API sean accesibles.

- **Configuración del Entorno Servidor:** Es necesario tener un servidor web activo (ej. Apache, provisto por XAMPP/WAMP/MAMP), que debe estar ejecutando los servicios de **Apache** y **MySQL**.
- **Ubicación de Archivos y Acceso:** Los archivos PHP de la API (`nuevo_api.php`, `listar_imagenes.php`, `luces_api.php`, etc.) deben

estar colocados en el directorio de servicio del servidor (típicamente `htdocs` o similar, dentro de una subcarpeta como `thermal_api`) para que puedan ser accedidos mediante llamadas HTTP desde la aplicación Flutter y el script Python.

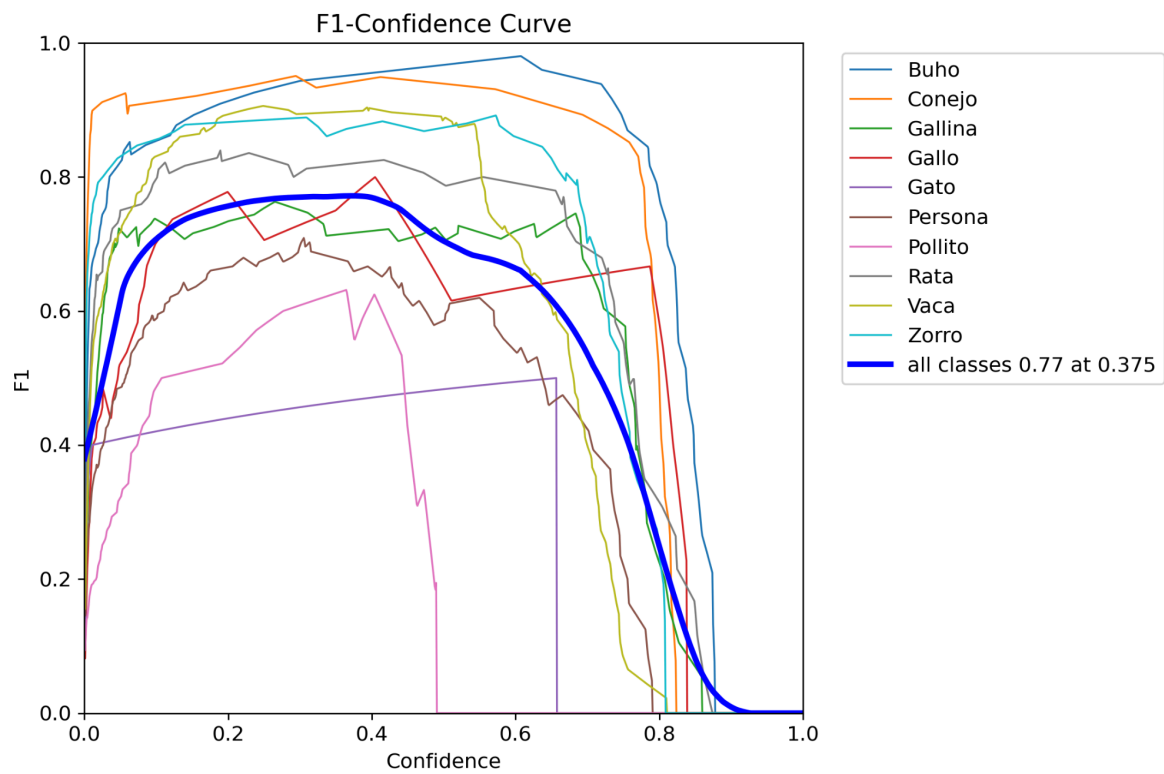
- **Gestión de Base de Datos:** La base de datos (presumiblemente **MySQL**) debe estar inicializada (ej. importando el esquema de `sensoresdb.sql`) para que los *scripts* de PHP puedan manejar el almacenamiento de los datos de los sensores y actualizar el estado de control (como el estado de las luces).

7. GRÁFICA DE LAS CONEXIONES

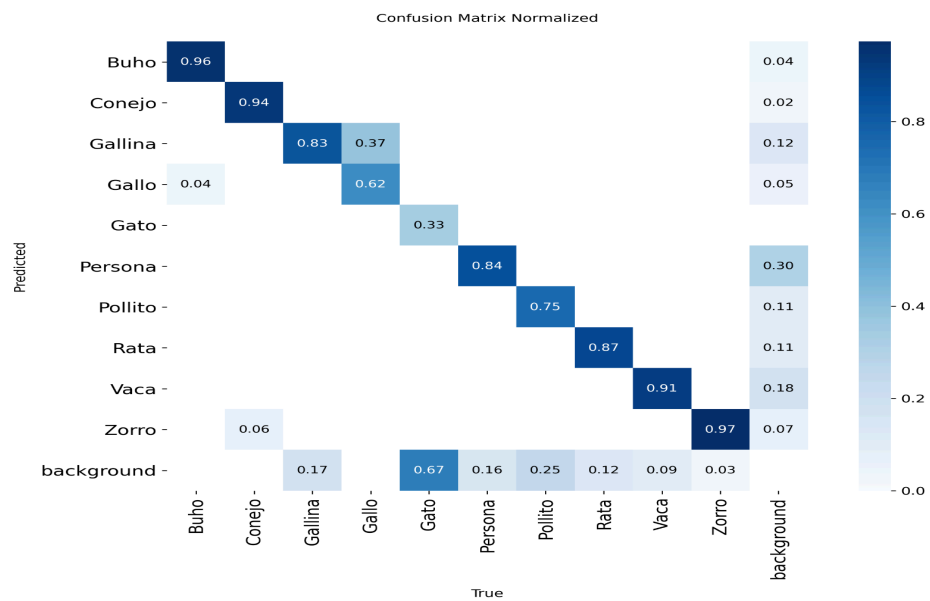


8. Gráficas y resultados del modelo

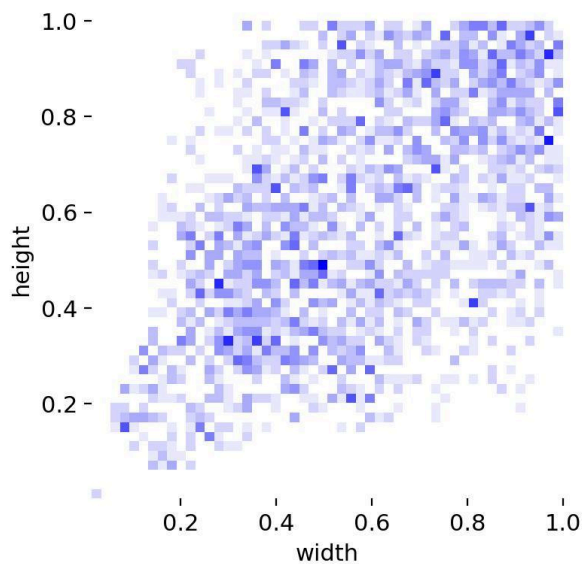
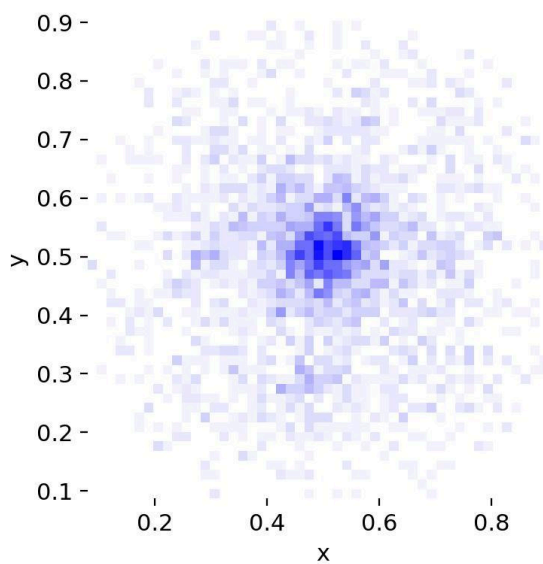
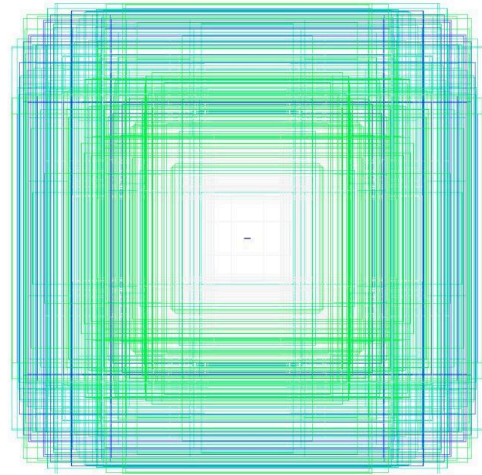
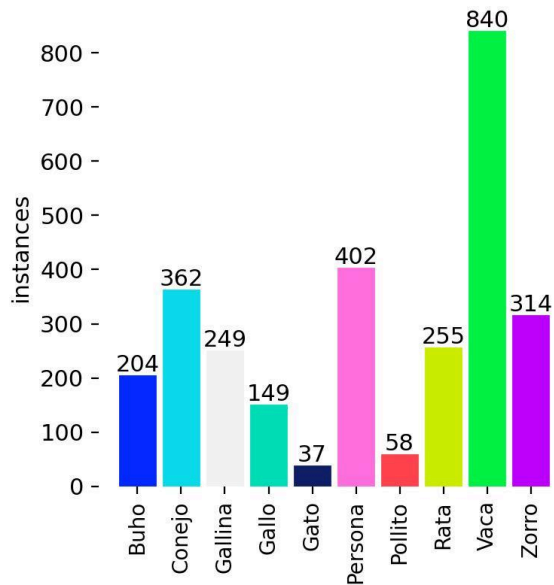
8.1 Curva de aprendizaje



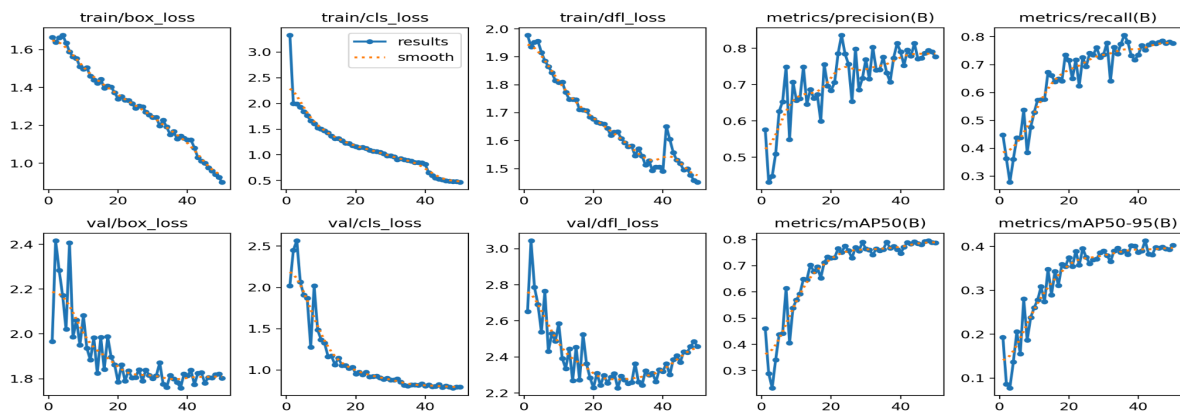
8.2 Matriz de confusión



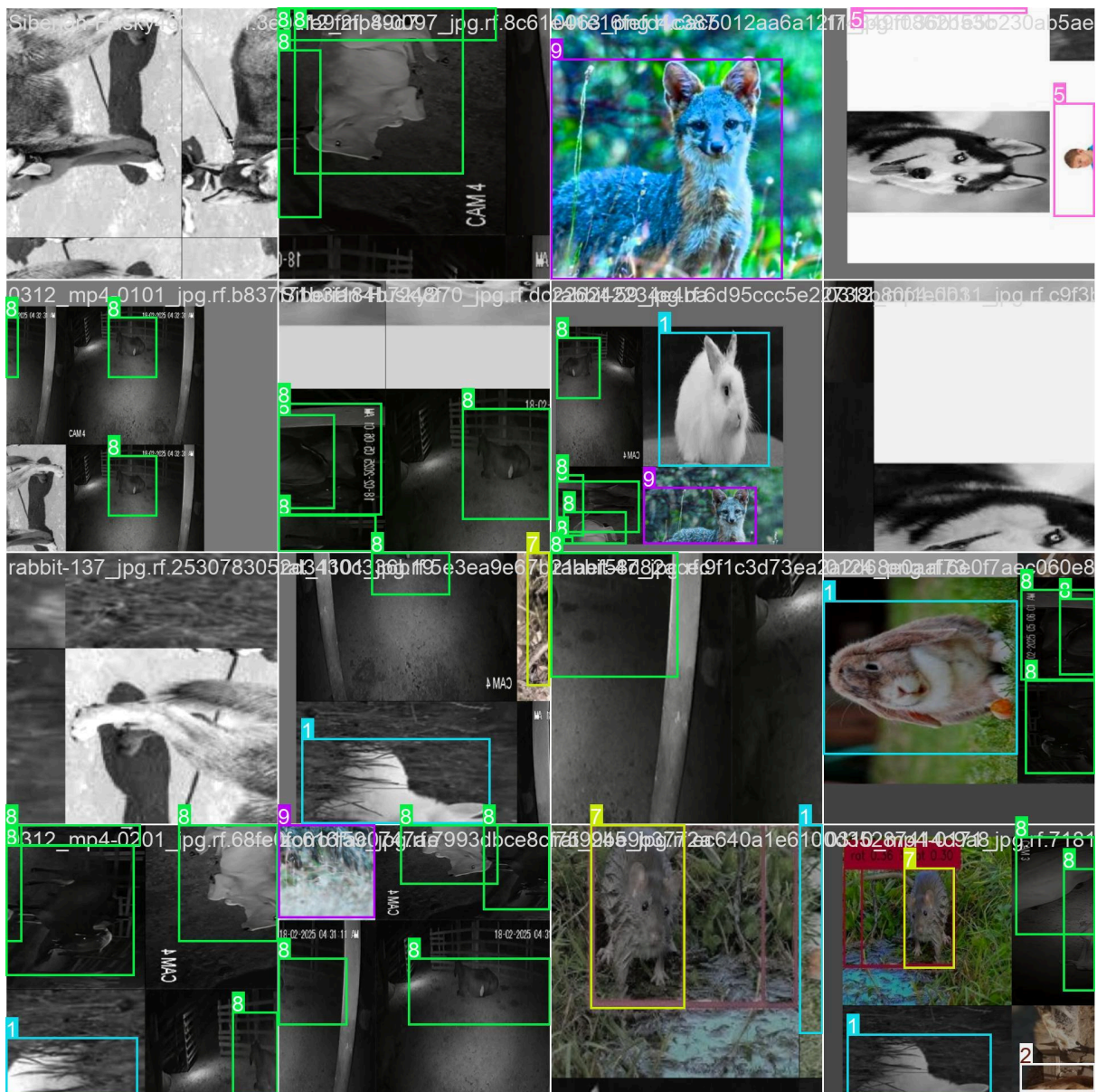
8.3 Labels



8.4. Resultados de train test y valid

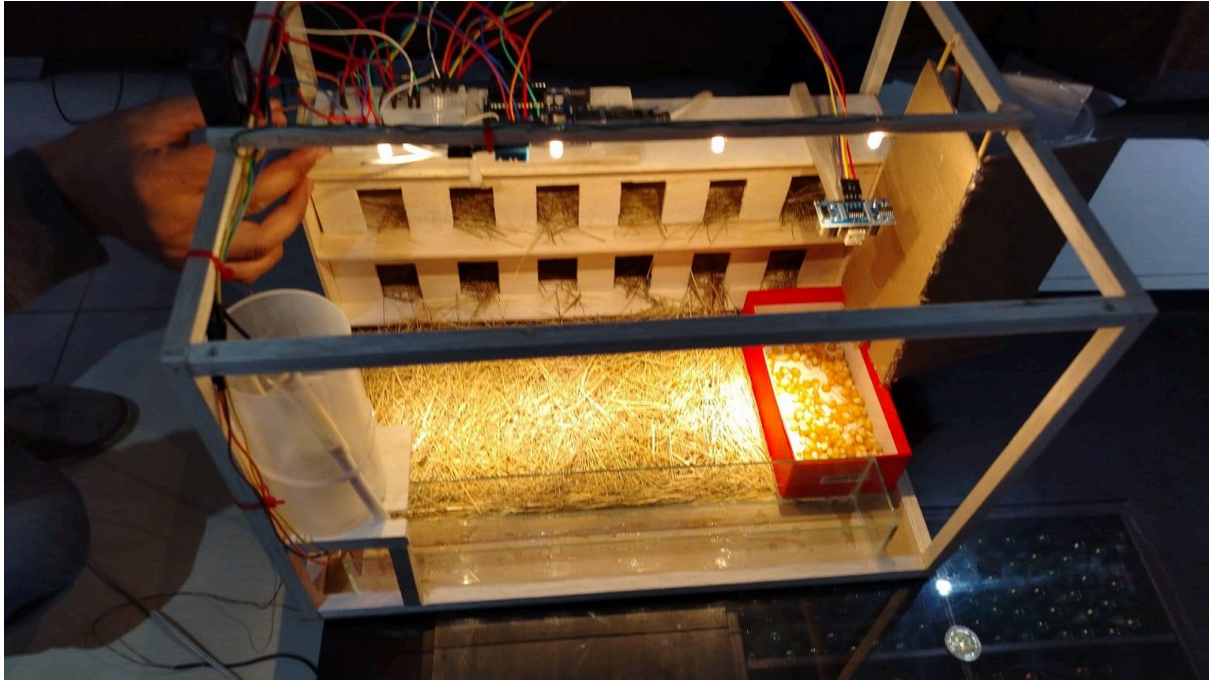


8.5 Prueba de imagen



9. Fotografías de la maqueta

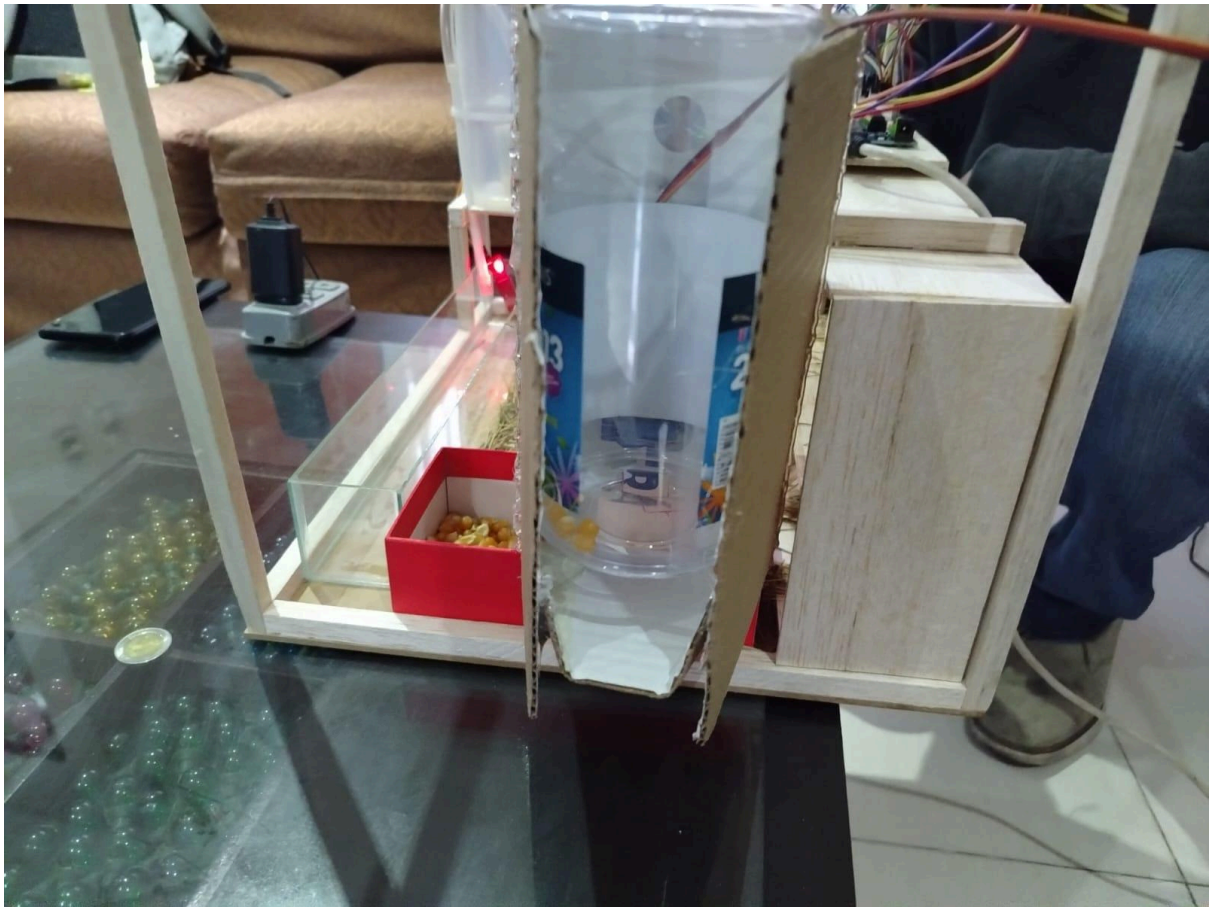
9.1 Prueba de foco



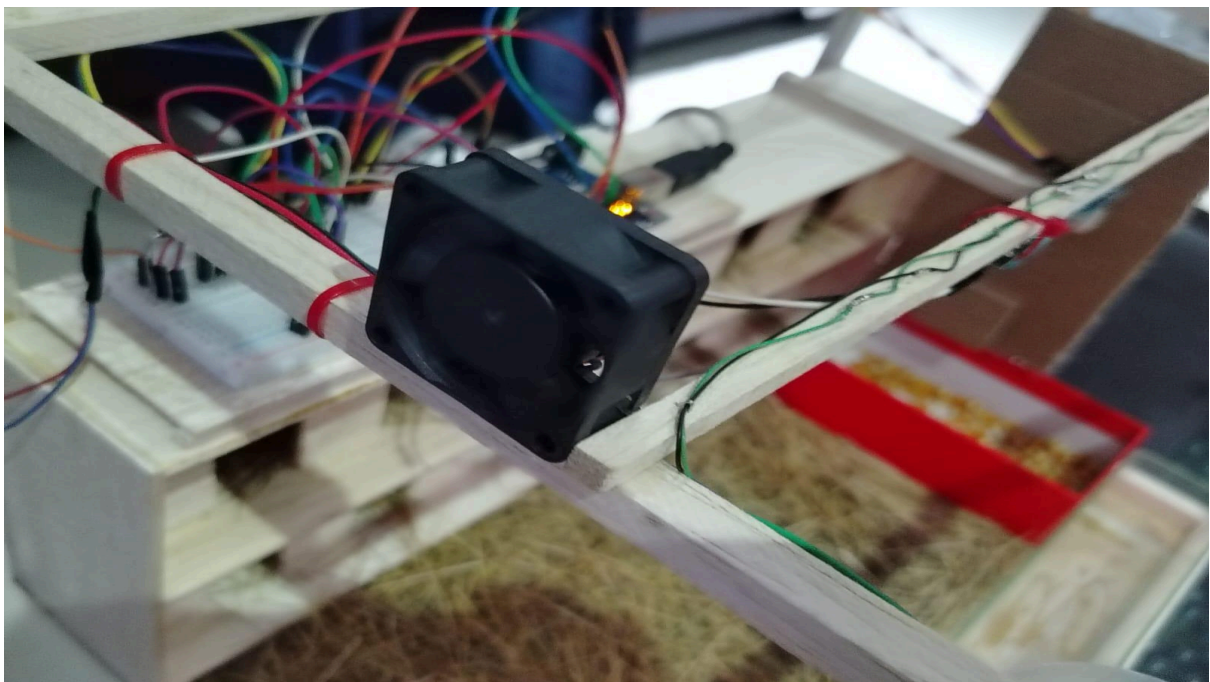
9.2. Prueba de comida



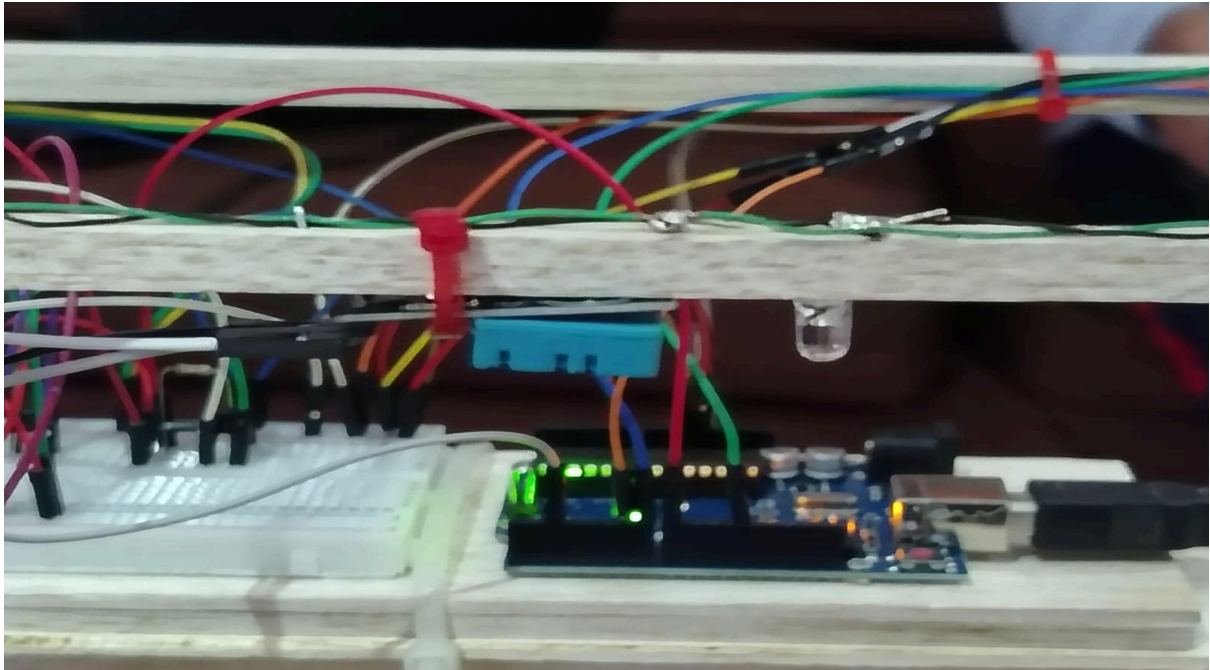
9.3. Bomba de agua



9.4 Ventilador

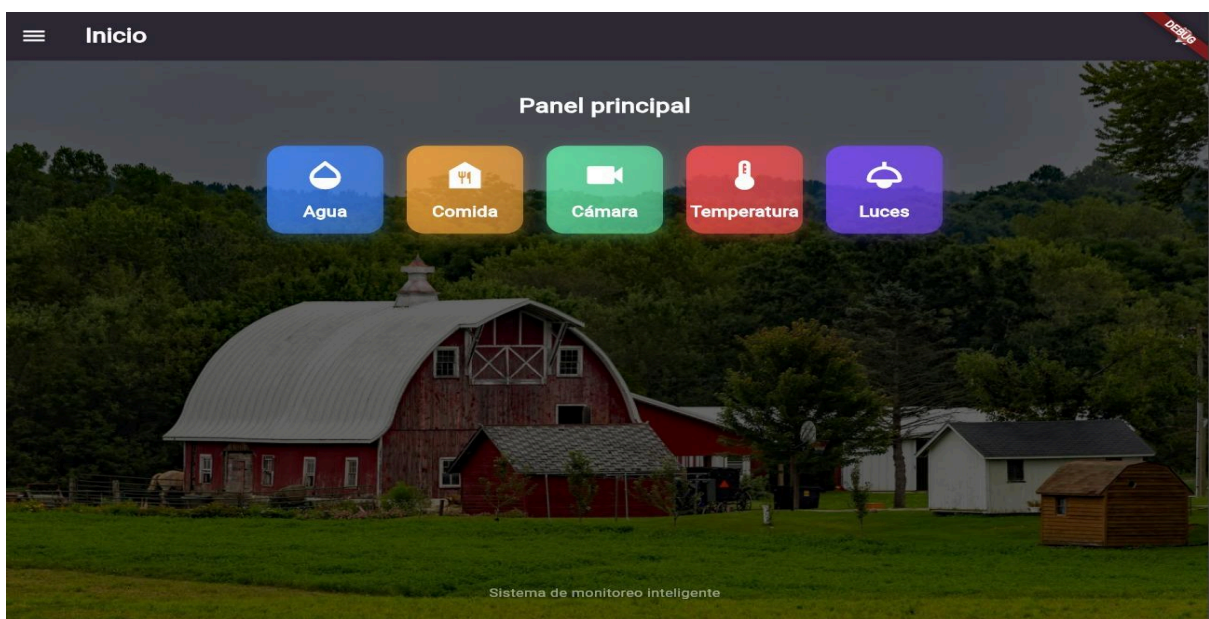


9.5 Temperatura



10. Capturas de la web

10.1. Home



10.2. Comedero

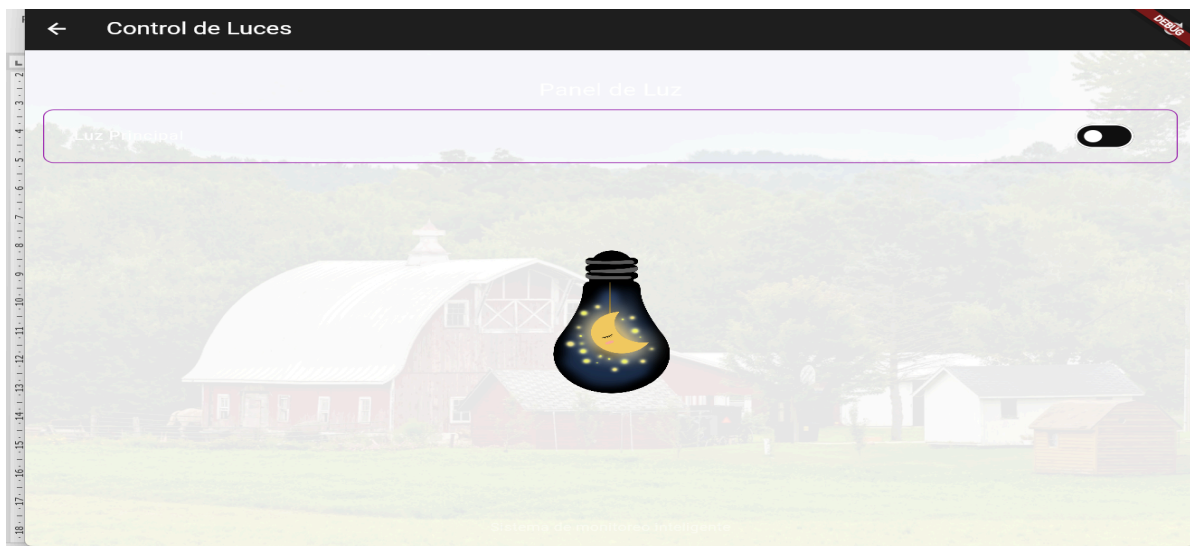


10.3. Bebedero

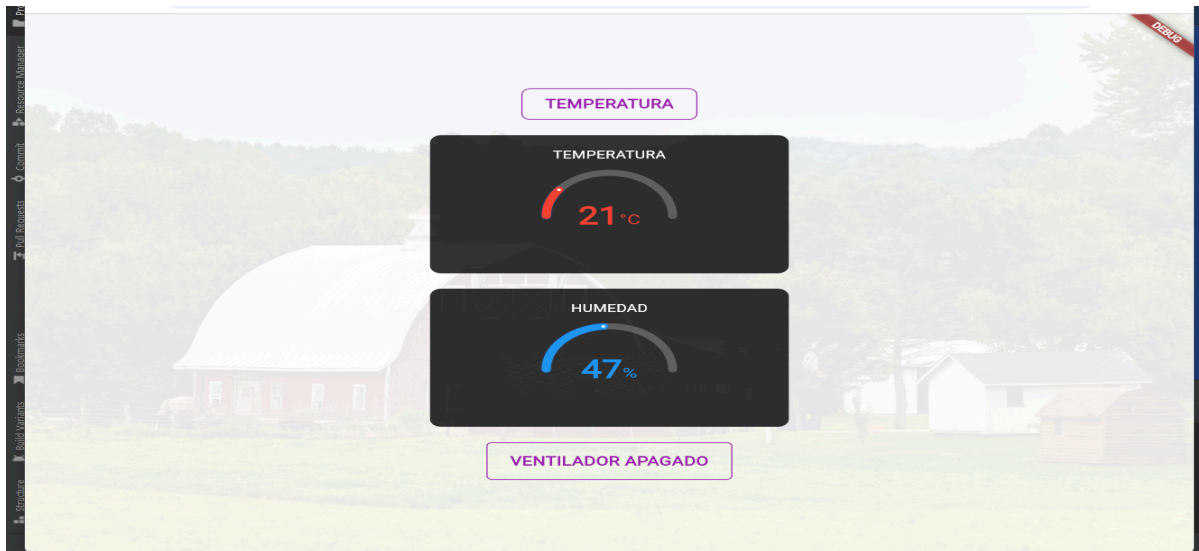




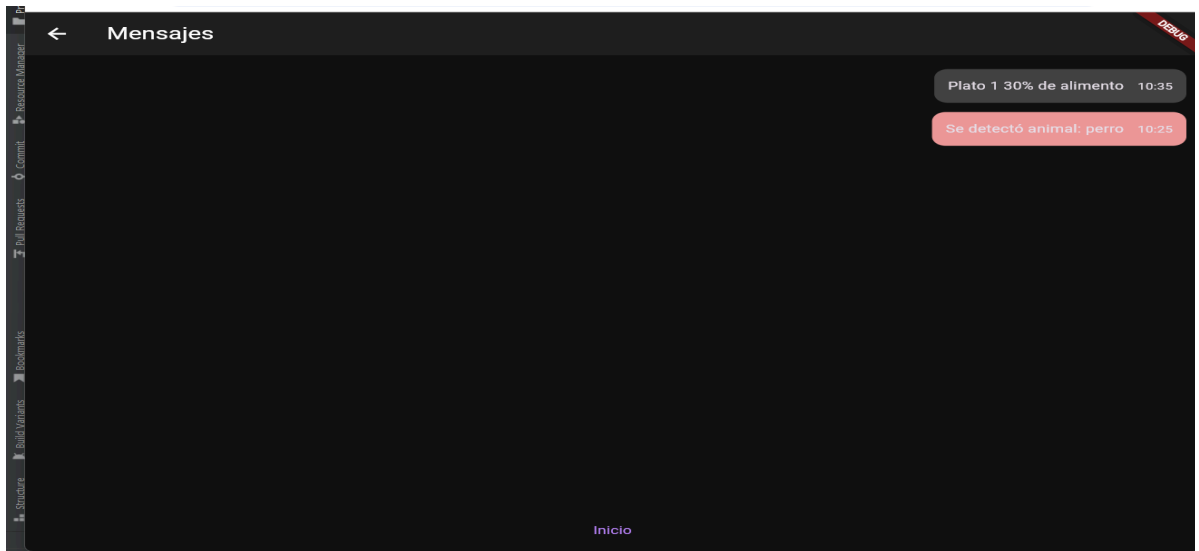
10.4. Luz



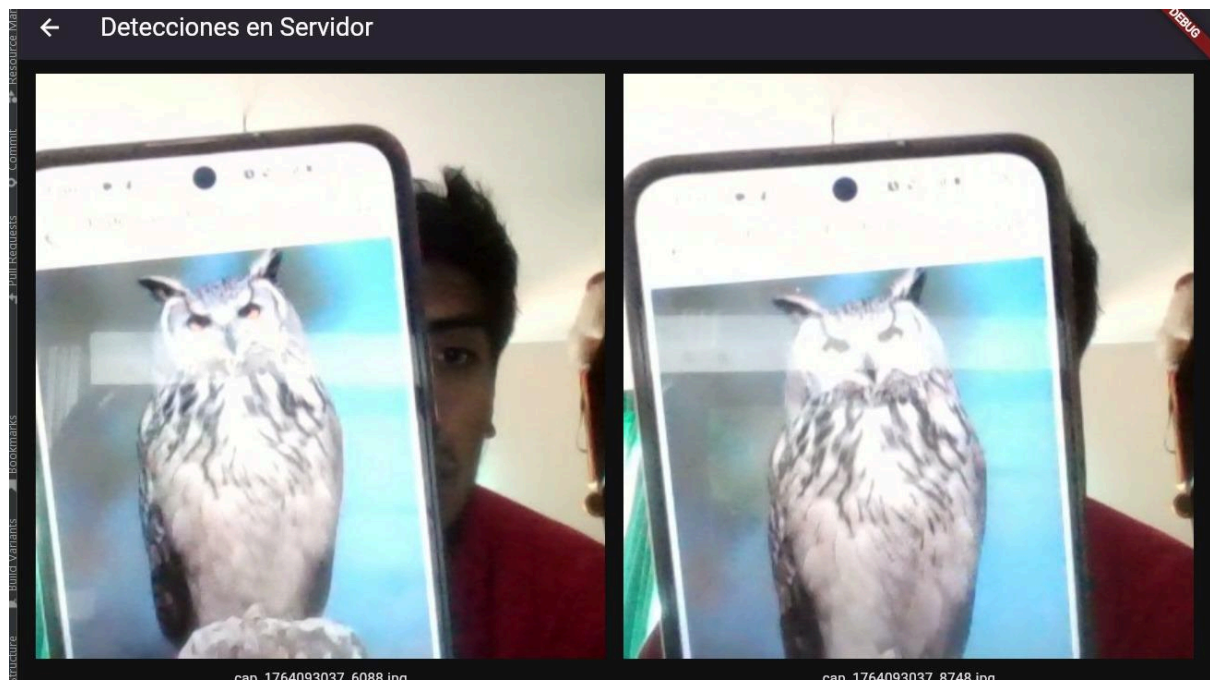
10.5. Temperatura



10.6. Notificaciones



10.7. Cámara



10.8. Detecciones



11. Conexiones

11.1. Conexión del sensor DHT11 (Temperatura / Humedad)

Componentes: DHT11 + resistencia de 10k

Conexión:

VCC del DHT11 → 5V del Arduino

GND del DHT11 → GND del Arduino

DATA del DHT11 → Pin digital 2 del Arduino

Colocar resistencia de 10k entre:

DATA ↔ VCC (resistor pull-up)

11.2. Conexión del sensor ultrasónico HC-SR04

Conexión:

VCC → 5V

GND → GND

TRIG → Pin 9

ECHO → Pin 10

11.3. Control de la mini bomba de agua (5V)

Se usa un transistor NPN + diodo + resistencia, igual que en tu esquema.

Conexión:

Colector del transistor → negativo de la bomba

Emisor del transistor → GND

Positivo de la bomba → 5V

Base del transistor:

Base → Resistencia (1k – 4.7k) → Pin digital 6 del Arduino

Protección:

Colocar diodo 1N4007/1N4148 paralelo a la bomba:

Ánodo → negativo

Cátodo → positivo

11.4. Control del ventilador de 5V

(lleva un transistor igual que la bomba)

Conexión:

Colector → negativo del ventilador

Emisor → GND

Positivo del ventilador → 5V

Base del transistor:

Base → Resistencia (1k – 4.7k) → Pin digital 7 del Arduino

Protección:

Diodo 1N4007 paralelo al ventilador.

11.5. LED indicador (opcional)

Conexión:

Ánodo del LED → Resistencia 220Ω → Pin 8 del Arduino

Cátodo → GND

11.6. Potenciómetro o sensor lineal (se ve en tu imagen superior)

Conexión típica (si es potenciómetro):

Extremo izquierdo → 5V

Extremo derecho → GND

Terminal central → Pin A0

11.7. Alimentación y GND

La placa Arduino alimenta todo con 5V a través del USB.

Todos los GND deben estar unidos (Arduino + bomba + ventilador + sensores).

11.8. Funcionamiento General

Componente	Controlado por	Función
DHT11	Lee T° y Humedad	Activa bomba o ventilador según condiciones
HC-SR04	Mide distancia	Puede servir para detectar nivel de agua
Mini Bomba	Transistor + pin 6	Riega cuando humedad es baja
Ventilador	Transistor + pin 7	Enfría cuando la temperatura es alta
LED	Pin 8	Alerta visual

11. Conclusión

El proyecto logró una **integración tecnológica eficaz** de tres componentes clave:

1. **Hardware (Arduino):** La capa física encargada de la sensórica y la actuación.
2. **Capa de Comunicación y Servidor:** Un *script* en Python se estableció como el puente vital, leyendo la información de los sensores del hardware y transmitiéndola a una API central de servidor (PHP/MySQL). Esta capa también gestiona el estado de los actuadores, como las luces, recibiendo comandos desde el móvil y enviándolos al hardware.
3. **Aplicación Móvil (Flutter):** La aplicación funciona como el panel de control centralizado, ofreciendo control remoto y visualización de datos en tiempo real.

Resultado y Valor Agregado:

El sistema garantiza un **monitoreo integral** de recursos (Agua, Comida) y condiciones ambientales (Temperatura). Permite el **control remoto** de funciones clave, como las luces, e incluye una funcionalidad de **vigilancia y registro de Detecciones** de eventos.

En resumen, el proyecto ha establecido una solución de Internet de las Cosas (IoT) completamente automatizada que mejora la eficiencia operativa del gallinero y contribuye directamente a garantizar el bienestar animal.

12. Bibliografía

Anjum, F. A., Ahmad, H., & Ahmad, N. (2025). IoT-Based Smart Agriculture Monitoring System. *International Journal of Advanced Research in Science, Communication and Technology*, 5(1). (Referencia sobre la aplicación de sistemas de monitoreo inteligente en el sector agropecuario).

Aosong Electronics. (s.f.). DHT22 (AM2302) Digital Temperature and Humidity Sensor Datasheet. (Documentación técnica de sensores clave utilizados para el monitoreo ambiental).

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems*. Pearson. (Referencia sobre los principios de los sistemas de bases de datos, como MySQL, utilizados para la persistencia de datos).

Google. (s.f.). Flutter: Build apps for any screen. Obtenido de <https://flutter.dev/>. (Documentación oficial de la plataforma de desarrollo de la aplicación móvil, **dispenser_app**).

Lee, E. A., & Seshia, S. A. (2017). *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*. MIT Press. (Referencia fundamental sobre la arquitectura de sistemas embebidos, crucial para el control con Arduino).

pySerial Developers. (s.f.). **Welcome to pySerial's documentation.** Obtenido de <http://pyserial.readthedocs.io/>. (Documentación de la librería de Python esencial para la comunicación serial bidireccional entre el *script* y el hardware).

Welling, L., & Thomson, L. (2003). *PHP and MySQL Web Development*. Sams Publishing. (Referencia para la implementación del backend, incluyendo la lógica de la API en PHP).